Lecture 18: Remoting II

Objectives:

- Learn how to configure remoting servers and clients declaratively
- Learn the difference between Server Activated Objects (SAO) and Client Activated Objects (CAO)
- Learn about the life-time of remoting objects
- Learn how to create event-generating remoting objects.

1. Declarative Configuration of Servers and Clients

We have seen how to configure remoting servers and clients in the last lecture. That type of configuration where the information about the server and the client is specified within the code is called **programmatic configuration**.

The disadvantage of making configuration programmatically is that if the server is moved to another machine, then the source code must be modified and recompiled.

An alternative configuration involves specifying the configuration information in the form of XML tags in a text file.

The server and the clients are then written so that they read the information from the text files. If there is any change in the information, the text files can be easily modified accordingly.

This type of configuration is called **declarative configuration**.

The following shows how to specify the configuration information in both the server and the client.

//File: MathServer2.config

```
<configuration>
 <system.runtime.remoting>
  <application>
   <channels>
    <channel ref="http" port="9090">
     <serverProviders>
         <provider ref="wsdl" />
      <formatter ref="soap" typeFilterLevel="Full" />
      <formatter ref="binary" typeFilterLevel="Full" />
     </serverProviders>
    </channel>
   </channels>
   <service>
```

```
          <wellknown mode="Singleton" type="MathClass, MathLibrary"
           objectUri="MyMathServer" />
    </service>
   </application>
 </system.runtime.remoting>
</configuration>
```

Note: If MathClass is in a namespace, Math, then you must include it in the type specification as: `type="Math.MathClass, MathLibrary"`
This is particularly important for users of Visual studio where namespace is always created by default.

The above configuration is fairly general. For the most part, you only need to change the port number and service part to reflect your remotable object.

The channel can be *http* or *tcp*. Tcp channels uses binary formatting by default, which is faster.

http by default uses soap (Simple Object Access Protocol) format. Soap is more verbose than binary, making it less efficient.

The advantage of http channel is that it can easily pass across firewalls.

A common compromise is to use http with binary format.

In the above we are defining both soap and binary format over http channel.

With the above configuration, the MathServer program reduces to few lines as shown below:

```
using System;
using System.Runtime.Remoting;

public class MathServer
{
     public static void Main() {
          RemotingConfiguration.Configure("MathServer2.config");
          Console.WriteLine("Math Server started, press Enter to
terminate...");
          Console.ReadLine();
     }
}
```

Note: You need to manually copy the DLL file to the location of the exe file for the server.

//File: MathClient2.config

```
<configuration>
 <system.runtime.remoting>
  <application>
   <channels>
    <channel ref="http" port="0">
     <clientProviders>
      <formatter ref="binary" />
     </clientProviders>
    </channel>
   </channels>
   <client>
        <wellknown type="MathClass, MathLibrary"
          url="http://localhost:9090/MyMathServer" />
   </client>
  </application>
 </system.runtime.remoting>
</configuration>
```

Again the only part that you may need to change is the client part.

```
using System;
using System.Runtime.Remoting;

public class MathClient
{
     public static void Main(string[] args)
     {
          RemotingConfiguration.Configure("MathClient2.config");
          MathClass math = new MathClass();

          if (math == null)
               Console.WriteLine("Could not locate Server");
          else
          {
               int a = 10; int b = 5;
               Console.WriteLine("{0} + {1} = {2}", a, b, math.Add(a,
b));
               Console.WriteLine("{0} - {1} = {2}", a, b,
math.Subtract(a, b));
               Console.WriteLine("{0} * {1} = {2}", a, b,
math.Multiply(a, b));
               Console.WriteLine("{0} / {1} = {2}", a, b, math.Divide(a,
b));
               Console.ReadLine();
          }
     }
}
```

2. Server-Activated Objects,SAO vs Client-Activated Objects, CAO

So far we have seen two modes of creating a remotable object: **SingleCall** and **Singleton**.

Both of these modes are **server activated**. They are server activated because when a client creates an instance of the remote object, that instance is not created immediately on the server side until a method is actually called.

Another mode available is **client-activated**. In this mode, an object is created the moment the client uses *new* to create the object.

Client-activated mode provides two advantages.

- First, it can be used to call a non-default constructor of the remote class. This is not possible in server-activated modes.
- Secondly, it provides a compromise in terms of life-time, between **SingleCall** and **Singleton**. In client-activated mode, the object created is unique for each client, moreover, the state of the object is maintained across multiple method calls as long as the client has a reference to the object.

```
using System;

public class StopWatch : MarshalByRefObject {
    DateTime start = DateTime.Now;

    public void Start () {
        start = DateTime.Now;
    }
    public int Stop () {
        return (int) ((DateTime.Now - start).TotalMilliseconds);
    }
}
```

//StopWatchServer.config

```
<configuration>
   <system.runtime.remoting>
      <application>
         <channels>
            <channel ref="http"  port="8080" />
         </channels>
         <service>
            <activated type="StopWatch, StopWatchLibrary"/>
         </service>
      </application>
```

```
    </system.runtime.remoting>
</configuration>
```

```
using System;
using System.Runtime.Remoting;

class StopWatchServer {
     static void Main () {
        RemotingConfiguration.Configure("StopWatchServer.config");
        Console.WriteLine ("Press <enter> to terminate...");
        Console.ReadLine ();
     }
}
```

// StopWatchClient.config

```
<configuration>
   <system.runtime.remoting>
      <application>
         <client url="http://localhost:8080">
            <activated type="StopWatch, StopWatchLibrary"/>
         </client>
         <channels>
            <channel ref="http" port="0">
               <serverProviders>
                  <formatter ref="soap" typeFilterLevel="Full"/>
                  <formatter ref="binary" typeFilterLevel="Full"/>
               </serverProviders>
            </channel>
         </channels>
      </application>
   </system.runtime.remoting>
</configuration>
```

```
using System;
using System.Runtime.Remoting;

class StopWatchClient {
     static void Main () {
         RemotingConfiguration.Configure("StopWatchClient.config");
         StopWatch sw = new StopWatch ();
         sw.Start ();

         Console.WriteLine ("Press Enter to show elapsed time...");
         Console.ReadLine ();

         Console.WriteLine (sw.Stop () + " millseconds");
           Console.ReadLine();
     }
}
```

Notice the difference in the configuration of client activated modes. The property, *activated*, is used for both client and server. Also

there is no need for URI in the server.

3. Object life time & Lifetimes Leases

The lifetime of a **Singleton server-activated** object and that of a **client-activated** object are controlled by leases.

A lease is an object that implements the **ILease** interface defined in the **System.Runtime.Remoting.Lifetime** namespace.

The ILease interface has the following properties:

Property	**Description**
InitialLeaseTime	Length of time following activation that the object lives if it receives no method calls
RenewOnCallTime	Minimum value that the CurrentLeaseTime is set to each time the object receives a call
CurrentLeaseTime	Amount of time remaining before the object is deactivated if it does not receive a method call

The default for **InitialLeaseTime** is 5 minutes, and the default for **RenewOnCallTime** is 2 minutes. However, you can override these defaults using either declarative configuration or programmatically as shown by the following:

```
using System;
using System.Runtime.Remoting.Lifetime;

public class RemotableClass : MarshalByRefObject {
    public override object InitializeLifetimeService ()     {
        ILease lease = (ILease) base.InitializeLifetimeService ();

        if (lease.CurrentState == LeaseState.Initial)  {
            lease.InitialLeaseTime = TimeSpan.FromMinutes (20);
            lease.RenewOnCallTime = TimeSpan.FromMinutes (10);
        }
        return lease;
    }
  ...
}
```

or

```
<configuration>
  <system.runtime.remoting>
    <application>
```

```
      <lifetime leaseTime="20M" renewOnCallTime="10M" />
    </application>
  </system.runtime.remoting>
</configuration>
```

Other suffixes you can use are : D for days, H for hours or S for seconds (the default).

To make an object to remain-active continuously, you can override the InitializeLifeTimeServices to return null as follows:

```
using System;

public class Foo : MarshalByRefObject {
    public override object InitializeLifetimeService ()
    {
        return null;
    }
  ...
}
```

4. Event generating remotable objects.

In addition to calling methods of a remotable objects, a client can also register with an event of a remotable object so that it is notified when such an event occurs.

In this way, it is easy to create a broadcasting system using remoting. All clients register with the remotable object's events. Each time the event occurs – example receiving a message from a client, all registered clients are notified.

To achieve this, we need to do the following additional settings:

- For the server, in addition to creating a service channel, it must also create a client channel through which it can notify clients about an event.
- For the clients, in addition to creating a client channel, it must also create a server channel through which it can be notified about an event.
- In addition to the DLL for the remotable object, the server must also have the code of the client. It needs this to bind its event with the method defined in the client which is used to register for the event.

- Finally, the method that is invoked by the server when the event occurs should be tagged with the **[OneWay]** attribute. This is a shorthand for OneWayAttribute, which is defined in the **System.Runtime.Remoting.Messaging** namespace.

  Calls to one way methods execute **asynchronously**. Moreover, the caller will not be notified of the result of the call. Such methods are called **fire-and-forget**.

Example:

The following example shows how to write a simple chat system using event-generating remotable object.

The remotable class:

```
using System;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Messaging;

public delegate void MessageHandler(string msg);

public class ChatClass : MarshalByRefObject
{
     public event MessageHandler MessageSender;

    public override object InitializeLifetimeService ()
    {
        return null;
    }

     [OneWay]
     public void SendMessage(string msg) {
          if (MessageSender != null)
               MessageSender(msg);
     }
}
```

The Chat Client:

```
using System;
using System.Windows.Forms;
using System.Runtime.Remoting;
using System.ComponentModel;

namespace MyFormProject
{
     public class ChatClient : System.Windows.Forms.Form
     {
          private System.Windows.Forms.TextBox inBox;
          private System.Windows.Forms.Button sendBt;
```

```
            private System.Windows.Forms.GroupBox groupBox;
            private System.Windows.Forms.GroupBox groupBox2;
            private System.Windows.Forms.TextBox outBox;

            private MessageHandler handler;
            private ChatClass chatObject;

            public ChatClient()
            {
                  InitializeComponent();
                try {
                    RemotingConfiguration.Configure
("ChatClient.config");

                    chatObject = new ChatClass();

                    handler = new MessageHandler(OnNewMessage);
                    chatObject.MessageSender +=  handler;
                 }
                catch (Exception ex) {
                    MessageBox.Show (ex.Message);
                    Close ();
                 }
            }

            void InitializeComponent() {
                  //deleted
            }

            [STAThread]
            public static void Main(string[] args)
            {
                  Application.Run(new ChatClient());
            }

            public void OnSendClicked(object sender, System.EventArgs
e)
            {
                  chatObject.SendMessage(outBox.Text);
            }

            public void OnNewMessage(string msg) {
                  inBox.Text += msg + "";
            }
          protected override void OnClosing (CancelEventArgs e)
           {
              // Disconnect event handler before closing
              base.OnClosing (e);
              chatObject.MessageSender -= handler;
           }
      }
  }
```